The Four Pillars of OOP

The core architectural design guidelines for resilient software engineering

The Four Pillars Overview

Encapsulation

Hiding data inside the object and exposing it only via methods and properties.

Abstraction

Hiding complex engine design logic behind simple interfaces.

Inheritance

Reusing properties and methods from existing base parent classes.

Polymorphism

Allowing custom derived classes to treat or morph shared behaviors differently.

1. Encapsulation: Data Shielding

Bundling state (fields) and behavior (methods) together into a class while restricting raw direct data access.

- ➔ Protects internal object state from illegal outside changes.
- ➔ Fields are set to `private`.
- ➔ Access is exposed safely using public C# **Properties**.

```
public class BankAccount {
    private double _balance; // Protected field

    public double Balance { // Property gateway
        get => _balance;
        set { if(value >= 0) _balance = value; }
    }
}
```

2. Abstraction: Complexity Reduction

```
public abstract class DatabaseConnector {
    // Subclasses handle implementation details
    public abstract void Connect();

    // Simple exposed method interface
    public void Sync() {
        Connect();
        /* sync logic */
    }
}
```

Hiding the messy, background implementation details and exposing only what is essential for interaction.

- ➔ Reduces the cognitive load on developers using your software modules.
- ➔ Achieved using C# `abstract` classes, methods, and `interface` definitions.

3. Inheritance: Code Reuse

Creating a new child class that automatically inherits all state fields and behavioral traits from a parent base class.

- ➔ Establishes an **"Is-A"** relationship hierarchy (e.g., a `Dog` is-a `Animal`).
- ➔ Promotes clean reuse of code assets without duplication.

```
public class User {
    public string Username { get; set; }
}

// Admin inherits from User
public class Admin : User {
    public void DeleteUser() { /* logic */ }
}
```

4. Polymorphism: Dynamic Identity

```
public class Shape {
    public virtual void Draw() => Console.Write("Shape");
}

public class Circle : Shape {
    public override void Draw() => Console.Write("Circle");
}

// Usage: Shape s = new Circle(); s.Draw(); // Prints "Circle"
```

The ability of an object to take on many forms. It allows a parent class reference to automatically trigger unique child behaviors during runtime execution.

- ➔ **Method Overriding:** Dynamic runtime change (using `virtual` and `override` keyword sets).
- ➔ Improves code modularity and decoupling.